

eQMS - API design

1. Overview

This document specifies the API surface for the Document Management Platform.

All endpoints are:

- **REST over HTTPS**
- **JSON-based**, except file upload/download
- **Authenticated via OIDC tokens**
- **Tenant-scoped**

All *modifying* operations (Create/Update/Delete) generate transactional audit events stored in the **Outbox** table.

All *read* operations generate asynchronous audit events published to a **Durable Queue**.

2. Authentication & Authorization

2.1 Authentication

- All APIs require a valid OIDC Bearer Token.
- JWT must include:
 - `sub` (user identifier)
 - `tenant_id` claim
 - `roles` array
 - `permissions` or `scope` (optional)

2.2 Authorization

- RBAC + ABAC enforced at the API layer.
- Common permissions:
 - `document.read`
 - `document.write`
 - `document.delete`
 - `compliance.audit.read` (Compliance Only)

3. Error Handling

Standard error envelope:

```
1 {  
2   "error": {
```

```
3     "code": "string",
4     "message": "string",
5     "details": {}
6   }
7 }
8
```

Common error codes:

- Unauthorized
- Forbidden
- NotFound
- ValidationFailed
- Conflict
- InternalError

4. API Endpoints

4.1 Document API

4.1.1 Create Document

POST /api/v1/documents

Creates a new document and generates a **transactional audit event** (CUD).

Request

```
1 {
2   "title": "Label Protocol V2",
3   "description": "Labeling procedure",
4   "metadata": {
5     "classification": "Controlled",
6     "tags": ["labelling", "v2"]
7   },
8   "contentBase64": "...."
9 }
10
```

Responses

- **201 Created**
- **Audit:** Outbox entry generated in same DB transaction

4.1.2 Get Document Metadata

GET /api/v1/documents/{documentId}

Returns metadata only.

- **Audit:** READ access logged asynchronously → queue.

Response

```
1 {  
2   "documentId": "doc-123",  
3   "title": "Label Protocol V2",  
4   "version": 3,  
5   "classification": "Controlled",  
6   "tags": ["labelling", "v2"],  
7   "createdAt": "2024-05-03T12:00:00Z"  
8 }  
9
```

4.1.3 Get Document Content

GET /api/v1/documents/{documentId}/content

Streams file content.

- **Audit:** READ logged asynchronously, including fieldsReturned.

Response

- Binary stream
- Headers:
 - Content-Type: application/pdf
 - Content-Disposition: attachment

4.1.4 Update Document Metadata

PUT /api/v1/documents/{documentId}

Modifies metadata, creates a new version if configured.

- **Audit:** Outbox entry created in same transaction.

Request

```
1 {  
2   "title": "Label Protocol V3",  
3   "metadata": {  
4     "classification": "Controlled",  
5     "tags": ["label", "v3"]  
6   }  
7 }  
8
```

Response

- 200 OK

4.1.5 Delete Document

DELETE /api/v1/documents/{documentId}

Soft delete.

- **Audit:** Outbox entry generated.

Response

- 204 No Content

4.2 Document Version API

4.2.1 List Document Versions

GET /api/v1/documents/{documentId}/versions

Returns all historical versions of a document.

Query Parameters (optional)

- startDate / endDate — filter by creation timestamp
- author — filter by user
- limit / offset — pagination

Response

```
1  [
2    {
3      "version": 1,
4      "createdAt": "2024-01-01T12:00:00Z",
5      "author": "user-123",
6      "description": "Initial release"
7    },
8    {
9      "version": 2,
10     "createdAt": "2024-03-15T09:45:00Z",
11     "author": "user-456",
12     "description": "Updated labeling info"
13   }
14 ]
15
```

4.2.2 Get Historical Document Metadata

GET /api/v1/documents/{documentId}/versions/{versionNumber}

Returns metadata for a specific version.

- **Audit:** READ logged asynchronously (fieldsReturned = metadata fields).

Response

```
1  {
2    "documentId": "doc-123",
3    "version": 2,
4    "title": "Label Protocol V2",
5    "classification": "Controlled",
6    "tags": ["labelling", "v2"],
7    "createdAt": "2024-03-15T09:45:00Z",
8    "author": "user-456",
9    "description": "Updated labeling info"
10 }
11
```

4.2.3 Get Historical Document Content

GET /api/v1/documents/{documentId}/versions/{versionNumber}/content

Streams file content for a historical version.

- **Audit:** READ logged asynchronously (fieldsReturned = content + metadata fields).

4.4 Tenant API

Create Tenant

POST /api/v1/tenants

Registers a new tenant in the system. Only admin/system operators can perform this action.

Request

```
1 {  
2   "name": "Acme Pharmaceuticals",  
3   "contactEmail": "admin@acme.com",  
4   "defaultDataClassification": "Controlled"  
5 }  
6
```

Response

- 201 Created
- Returns:

```
1 {  
2   "id": "tenant-123",  
3   "name": "Acme Pharmaceuticals",  
4   "createdAt": "2025-11-15T12:34:56Z"  
5 }
```

Get Tenant

GET /api/v1/tenants/{tenantId}

Retrieve tenant details.

- **Audit:** READ logged asynchronously for compliance.

List Tenants

GET /api/v1/tenants

Returns all tenants visible to the caller (system admin only).

4.5 User API

Create User

POST /api/v1/tenants/{tenantId}/users

Creates a user within a tenant. Includes initial roles/permissions.

Request

```
1 {
2   "username": "john.doe",
3   "email": "john.doe@acme.com",
4   "roles": ["DocumentEditor", "DocumentViewer"],
5   "permissions": ["document.read", "document.write"]
6 }
7
```

Response

- 201 Created

- Returns:

```
1 {
2   "userId": "user-123",
3   "username": "john.doe",
4   "roles": ["DocumentEditor", "DocumentViewer"],
5   "createdAt": "2025-11-15T12:34:56Z"
6 }
7
```

- **Audit:** Outbox entry generated in same transaction.

Get User

GET /api/v1/tenants/{tenantId}/users/{userId}

Retrieve user details.

- **Audit:** READ logged asynchronously → queue.

Update User

PUT /api/v1/tenants/{tenantId}/users/{userId}

Update user roles or permissions.

Request

```
1 {
2   "roles": ["DocumentViewer"],
3   "permissions": ["document.read"]
4 }
5
```

- **Audit:** Outbox entry generated.

4.2.4 Delete User

DELETE /api/v1/tenants/{tenantId}/users/{userId}

Soft delete or disable user.

- **Audit:** Outbox entry generated.

4.2.5 List Users

GET /api/v1/tenants/{tenantId}/users

Return all users in a tenant.

- **Audit:** READ logged asynchronously → queue.

4.3 Roles & Permissions API

4.3.1 Create Role

POST /api/v1/tenants/{tenantId}/roles

Define a new role for the tenant.

Request

```
1 {  
2   "roleName": "DocumentApprover",  
3   "description": "Can approve documents for release",  
4   "permissions": ["document.read", "document.approve"]  
5 }  
6
```

Response

- 201 Created
- Returns role details and assigned permissions.
- **Audit:** Outbox entry generated.

Get Role

GET /api/v1/tenants/{tenantId}/roles/{roleName}

Retrieve role details and permissions.

- **Audit:** READ logged asynchronously.

Update Role

PUT /api/v1/tenants/{tenantId}/roles/{roleName}

Update permissions for a role.

- **Audit:** Outbox entry generated.

Delete Role

DELETE /api/v1/tenants/{tenantId}/roles/{roleName}

Remove a role.

- **Audit:** Outbox entry generated.

GET /api/v1/tenants/{tenantId}/roles

Return all roles in a tenant.

- **Audit:** READ logged asynchronously → queue.

5. Access Logging (READ Audit)

5.1 How the API logs

For every read endpoint (metadata or content), the API pushes this message to the queue:

```
1 {  
2   "eventId": "uuid",  
3   "tenantId": "uuid",  
4   "timestamp": "2025-10-23T12:34:56Z",  
5   "userId": "u-123",  
6   "action": "Document.View",  
7   "objectType": "Document",  
8   "objectId": "doc-123",  
9   "fieldsReturned": ["title", "version"],  
10  "sourceIp": "1.2.3.4"  
11 }  
12
```

API does **not** wait for audit persistence.

6. Compliance Audit API (Read Only) - Next phase

6.1 Query Audit Index

GET /api/v1/audit

Returns records from the metadata index (NOT blob data).

Query parameters

- tenantId={uuid}
- start={ISO}
- end={ISO}
- action={string}
- objectType={string}
- objectId={string}
- actor={string}
- limit=200

Example:


```
1 GET /api/v1/audit?tenantId=...&start=2025-01-01&end=2025-01-
2 31&action=Document.View
```

Response

```
1 {
2   "items": [
3     {
4       "eventId": "uuid",
5       "timestamp": "2025-01-12T14:33:45Z",
6       "action": "Document.Update",
7       "objectType": "Document",
8       "objectId": "doc-22",
9       "actor": "user-5",
10      "blobUri": "blob://a/b/c.json1",
11      "offset": 233
12    }
13  ]
14 }
15
```

6.2 Export Audit Trail (Blob-based)

POST /api/v1/audit/export

Generates a CSV, PDF, or JSON export from underlying immutable blob data.

Request

```
1 {
2   "tenantId": "uuid",
3   "start": "2025-01-01T00:00:00Z",
4   "end": "2025-02-01T00:00:00Z",
5   "format": "csv"
6 }
7
```

Response

- 202 Accepted
- Returns jobId (export runs in background)

Download Result

GET /api/v1/audit/export/{jobId}/file

Compliance-only.

6.3 Get Raw Audit Event

GET /api/v1/audit/{eventId}

Fetches a single event by reading from blob using metadata offset.

7. System Internal APIs (Worker Side)

7.1 Outbox Forwarder Polling

Not exposed publicly, worker executes:

```
1 GET /internal/outbox/unprocessed
2 POST /internal/outbox/mark-processed
3
```

If these are implemented via direct DB connection, these internal APIs may not exist.

7.2 Queue Handler Summary

Worker receives:

```
1 {
2   "eventId": "uuid",
3   "tenantId": "...",
4   "timestamp": "...",
5   "payload": { ... }
6 }
7
```

Then batches → writes to blob → writes metadata record → marks complete.

8. Audit Event Model

Unified audit event JSON:

```
1 {
2   "eventId": "uuid",
3   "tenantId": "uuid",
4   "timestamp": "2025-11-15T12:34:56Z",
5   "actor": {
6     "userId": "u-123",
7     "roles": ["Editor"]
8   },
9   "action": "Document.Update",
10  "objectType": "Document",
11  "objectId": "doc-123",
12  "requestId": "guid",
13  "correlationId": "guid",
14  "sourceIp": "4.3.2.1",
15  "requestUrl": "/api/v1/documents/doc-123",
16  "payload": {
17    "before": {},
18    "after": {}
19  },
20  "fieldsReturned": ["documentId", "title", "version",
21    "classification", "tags"],
22  "system": {
23    "application": "DocumentService",
24    "version": "1.0.0"
25  }
26 }
```

9. Multi-Tenancy Rules

- Every request must include `tenant_id` claim.
- API rejects multi-tenant attempts (e.g., filter without `tenantId`).
- Audit data strictly partitioned by `tenantId`.
- Compliance roles may have cross-tenant access if allowed via SOP.

10. Performance Considerations

- List/search endpoints paginated.
- Download endpoints streaming.
- Audit endpoints limit results by default (e.g., 200 items).
- Export API offloads large results to background processing.

11. Non-Functional Requirements

- **Availability:** 99.9%
- **Latency:** < 150ms for standard API calls
- **Audit Log Latency:** < 60 seconds to appear in metadata index
- **Security:** OIDC, RBAC/ABAC, encrypted storage, key rotation
- **Immutability:** Blob WORM policy per regulatory requirements